SpecSnap: Android Application

Engineering Roadmap & Technical Specification

1. Executive Summary

SpecSnap is an open-source, production-grade Android application engineered for electronics makers, hardware hackers, and embedded systems engineers. Its core purpose is to identify physical surface-mount and through-hole circuit board components in real time and extract comprehensive technical specifications directly from visual data.

The application implements a high-performance hybrid processing architecture:

- **On-Device Lightweight Edge Vision:** Utilizes Tensor Flow Lite (YOLO/SSD quantized models) over a high-throughput CameraX pipeline to deliver real-time 30 FPS bounding box proposals for broad component classes (e.g., ICs, Resistors, Capacitors, Transistors, Headers).
- **Multimodal Cloud Reasoning:** Leverages the Gemini 1.5 Flash multimodal model via structured JSON output pipelines to execute high-fidelity Optical Character Recognition (OCR) on microscopic laser etchings, query component pinouts, parse datasheet operational parameters, and compute pin mapping topologies dynamically.

---

2. Architecture & Data Flow

The architecture is divided into a low-latency edge perception loop running locally on the device GPU/NPU and an on-demand cloud reasoning loop triggered by user interaction.+----------+

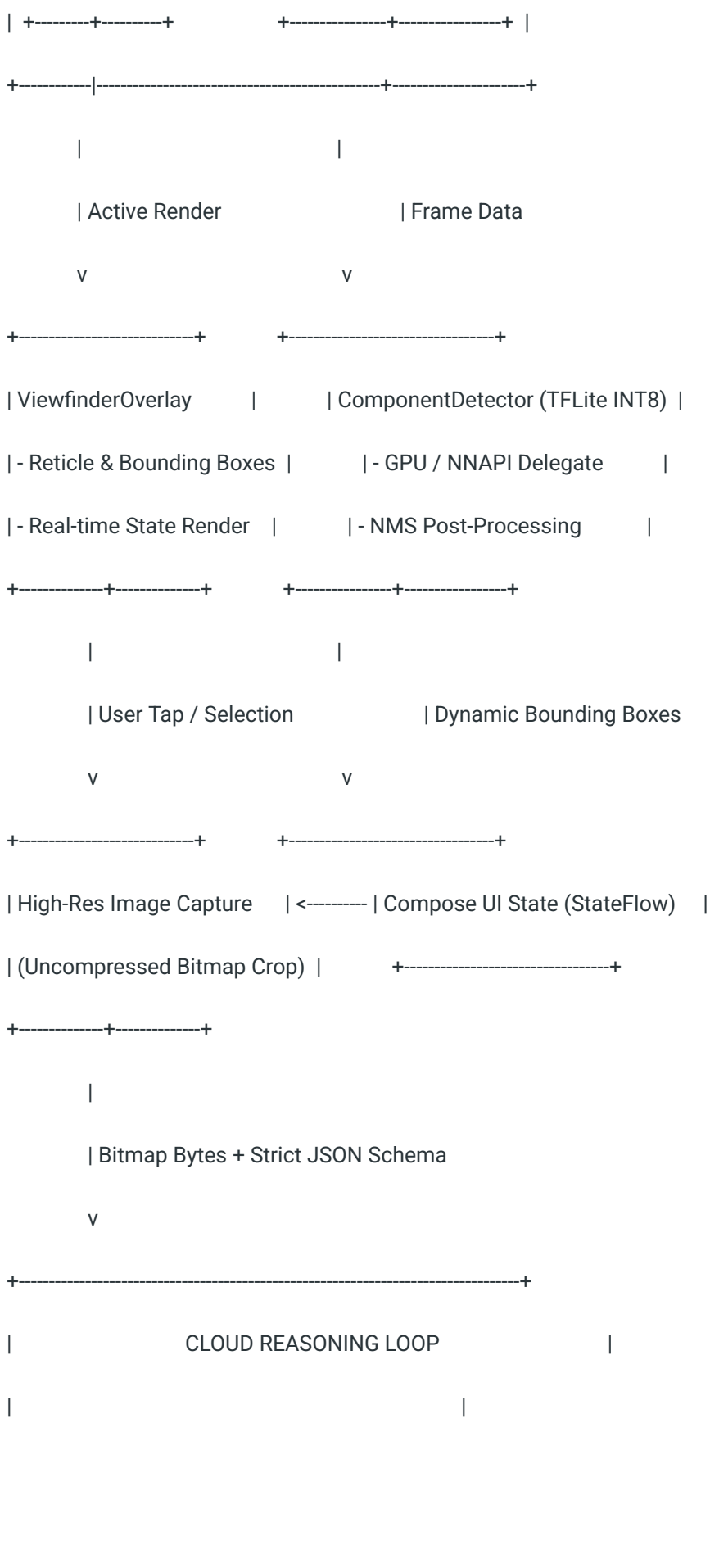
| CAMERAX PIPELINE |

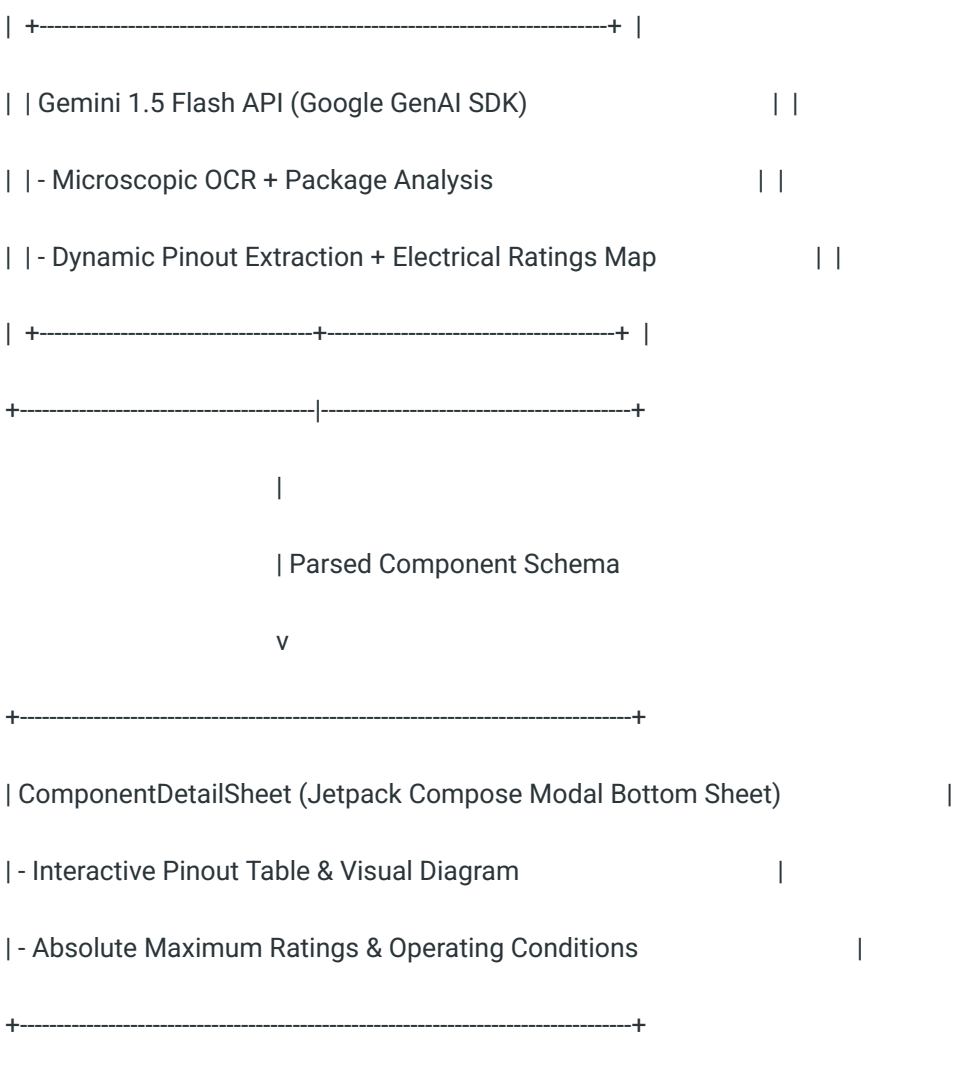
| |

| +----------+ YUV_420_888 +----------+ |

| | CameraX Preview | ----------> | ImageAnalysis Pipeline | |

| | (30 FPS Live) | | (Target Resolution: 640x640) | |


| +---------+---------+ +---------------+----------------+ |
+-----------|---+----------------------+
| |
| Active Render | Frame Data
v v
+----------------------------+ +----------------------------------+
ViewfinderOverlay		ComponentDetector (TFLite INT8)
- Reticle & Bounding Boxes		- GPU / NNAPI Delegate
- Real-time State Render		- NMS Post-Processing
+-------------+-------------+ +---------------+----------------+
| |
| User Tap / Selection | Dynamic Bounding Boxes
v v
+----------------------------+ +----------------------------------+
| High-Res Image Capture | <--------- | Compose UI State (StateFlow) |
| (Uncompressed Bitmap Crop) | +----------------------------------+
+-------------+-------------+
|
| Bitmap Bytes + Strict JSON Schema
v
+---+
| CLOUD REASONING LOOP |
| |



System Data Flow Sequence

1. **Continuous Edge Loop:** CameraX feeds `YUV_420_888` frames to `ImageAnalysis`. Frames are letterboxed, converted to TensorBuffers, and passed to `ComponentDetector` running TFLite with GPU delegation. Proposed bounding boxes are continuously emitted via `StateFlow` to the `ViewfinderOverlay`.
2. **Trigger & Crop:** On user tap (either manual button or directly on an edge bounding box), `ImageCapture` retrieves a full-resolution bitmap. The image utility crops the selected region matching the normalized target box without aspect ratio distortion.
3. **Cloud Reasoning:** The cropped bitmap is sent via `GeminiRepository` to Gemini 1.5 Flash using structured output mode (`responseMimeType = "application/json"`) enforced by a strict response schema.

4. **UI State Hydration:** The structured JSON response hydrates domain models (`ComponentResponse`), which open the `ComponentDetailSheet` displaying interactive pinouts, specs, and warnings.

---

3. Granular Step-by-Step Implementation Roadmap

Phase 1: Project Scaffolding & Environment Setup

Step 1.1: Project Initialization

- Initialize project via Android Studio Hedgehog/Iguana using **Empty Compose Activity** template.
- Configure `build.gradle.kts` (Project-level & App-level) with:
    - `minSdk = 26` (Android 8.0 Oreo to ensure comprehensive camera/hardware support).
    - `targetSdk = 35` / `compileSdk = 35`.
    - Enable Jetpack Compose with Kotlin compiler extension versions targeting Kotlin 2.0+.

Step 1.2: Dependency Configuration

Add the following core dependencies to `app/build.gradle.kts`:dependencies {

// AndroidX & Lifecycle

implementation(libs.androidx.core.ktx)

implementation(libs.androidx.lifecycle.runtime.ktx)

implementation(libs.androidx.activity.compose)

// Jetpack Compose (Material3)

implementation(platform(libs.androidx.compose.bom))

implementation(libs.androidx.compose.ui)

implementation(libs.androidx.compose.material3)

implementation(libs.androidx.compose.ui.tooling.preview)

// CameraX

```
    implementation(libs.androidx.camera.core)
    implementation(libs.androidx.camera.camera2)
    implementation(libs.androidx.camera.lifecycle)
    implementation(libs.androidx.camera.view)
    // Generative AI SDK
    implementation(libs.google.genai)
    // TensorFlow Lite & Support
    implementation(libs.tensorflow.lite)
    implementation(libs.tensorflow.lite.gpu)
    implementation(libs.tensorflow.lite.support)
    // Serialization & Coroutines
    implementation(libs.kotlinx.serialization.json)
    implementation(libs.kotlinx.coroutines.android)
}
```

Step 1.3: Secure API Key Management

- Configure `secrets-gradle-plugin` or manual `local.properties` injection in `app/build.gradle.kts`:

```
buildTypes {
    localProperties.getProperty("GEMINI_API_KEY")?.let { apiKey ->
        buildConfigField("String", "GEMINI_API_KEY", "\"$apiKey\"")
    }
}
```

- Ensure `.gitignore` contains `local.properties` to prevent accidental credential leaking.

Step 1.4: Manifest Permissions & Hardware Features

Define required camera and network permissions inside `AndroidManifest.xml`:<uses-permission android:name="android.permission.CAMERA" />

<uses-permission android:name="android.permission.INTERNET" />

<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

<uses-feature android:name="android.hardware.camera" android:required="true" />

<uses-feature android:name="android.hardware.camera.autofocus" android:required="false" />

Phase 2: Camera Pipeline & Viewfinder UI

Step 2.1: Runtime Permission Handler

- Implement `RequestCameraPermission` composable leveraging `rememberLauncherForActivityResult` with `ActivityResultContracts.RequestPermission()`.
- Render alternative UI state when permission is denied, providing direct links to application system settings.

Step 2.2: CameraPreview Composable

- Build a reusable `CameraPreview` UI block binding `PreviewView` to Android's `ProcessCameraProvider`.
- Bind lifecycle management to `LocalLifecycleOwner.current` to release resources when transitioning background states.

Step 2.3: ImageCapture & Touch-to-Focus

- Initialize `ImageCapture` with performance mode `CAPTURE_MODE_MINIMIZE_LATENCY` or `CAPTURE_MODE_MAXIMIZE_QUALITY`.
- Attach an `onTouch` listener to `PreviewView` translating raw `MotionEvent` coordinates into a CameraX `MeteringPoint` for precise auto-focus and auto-exposure metering.

Step 2.4: Viewfinder Overlay

- Build `ViewfinderOverlay` composable with `Canvas` layer rendering:
    - Centered reticle target for default manual scans.

- Animated scanning indicator line using `infiniteTransition`.
- Fallback manual trigger button positioned anchored at the bottom center.

Phase 3: Cloud Multimodal Pipeline (Fast MVP)

Step 3.1: Data Layer & Models

Define domain classes using `kotlinx.serialization` to enforce structural integrity during parsing:@Serializable

```
data class ComponentResponse(
    val partNumber: String,
    val manufacturer: String,
    val category: String,
    val description: String,
    val pinouts: List<PinoutEntry>,
    val electricalRatings: ElectricalRatings,
    val safetyAlerts: List<String> = emptyList()
)

@Serializable
data class PinoutEntry(
    val pinNumber: Int,
    val pinName: String,
    val function: String
)

@Serializable
```

```
data class ElectricalRatings(
    val minVoltage: String,
    val maxVoltage: String,
    val maxCurrent: String,
    val operatingTempRange: String
)
```

Step 3.2: GeminiRepository Implementation

- Instantiate `GenerativeModel` targeting `gemini-1.5-flash`.
- Configure `GenerationConfig` setting `responseMimeType = "application/json"`.
- Supply prompt engineering demanding strict structural extraction based on the provided JSON format.

Step 3.3: Image Cropping Utility

- Construct `BitmapCropUtils.cropRegion(source: Bitmap, targetBounds: RectF): Bitmap`.
- Map frame coordinates correctly between `PreviewView` aspect ratio scaling and underlying raw image resolution buffer.

Step 3.4: Event Pipeline Integration

- Wire trigger flow: User Tap -> Screen Coordinates -> Pixel Bitmap Extraction -> `GenerativeModel.generateContent()` -> JSON Parsing -> UI State update.

Step 3.5: Component Detail Sheet

- Create `ComponentDetailSheet` using Material3 `ModalBottomSheet`.
- Renders tabbed sections: **Overview**, **Pinout Table**, and **Electrical Specifications**.

Phase 4: On-Device Edge Detection (TFLite Integration)

Step 4.1: Edge Model Setup

- Obtain quantized INT8 TFLite model trained on electronics components (classes: `IC`, `Resistor`, `Capacitor`, `Transistor`, `Header`).
- Place `electronics_yolo_int8.tflite` in `app/src/main/assets/`.

Step 4.2: TFLite Inference Class

- Implement `ComponentDetector` wrapping `Interpreter`.
- Configure interpreter options enabling GPU delegate via `GpuDelegate()` falling back to NNAPI or multi-threaded CPU processing.
- Implement Non-Maximum Suppression (NMS) to clear overlapping target boxes.

Step 4.3: CameraX ImageAnalysis Pipeline

- Setup `ImageAnalysis.Builder` setting backpressure strategy to `STRATEGY_KEEP_ONLY_LATEST`.
- Convert `ImageProxy` frames (`YUV_420_888`) to scaled 640x640 `TensorBuffer` inputs.

Step 4.4: Dynamic Visual Bounding Boxes

- Expose detected bounding boxes through `StateFlow<List<DetectedComponent>>`.
- Draw dynamic boxes in `ViewfinderOverlay` canvas using scaled normalized coordinates.

Step 4.5: Direct Bounding Box Interaction

- Attach hit-testing logic in overlay: Tapping inside an edge-detected bounding box selects that region directly and triggers cloud analysis for that specific crop area.

Phase 5: Resistor Color Band Calculator & Offline Utilities

Step 5.1: Offline Calculation Engine

- Construct lookup engine for 4-band and 5-band through-hole resistor color decoding.
- Outputs nominal resistance value, multiplier, and standard tolerance percentage.

Step 5.2: Interactive Calculator UI

- Build `ResistorCalculatorScreen` using Jetpack Compose.

- Includes dropdowns or visual band selectors allowing engineers to verify manual calculations when offline.

Step 5.3: Offline Local Cache

- Set up a local Room Database (`ScanHistoryDatabase`) to persist previously scanned items, JSON metadata, and cropped images for fast offline recall.

Phase 6: Play Store Polish & Hardening

Step 6.1: Resilience & Error Handling

- Handle network drops, parsing exceptions, and low-confidence inference gracefully using clear `Snackbar` alerts or inline UI warnings.

Step 6.2: Settings & Customization

- Add a Settings screen allowing users to toggle Edge-Only processing mode or override the system default Gemini API Key with their personal Google AI Studio key.

Step 6.3: ProGuard & Shrinking Rules

- Add explicit keep rules in `proguard-rules.pro` for TFLite runtime, Serialization models, and Generative AI SDK structures.

Step 6.4: CI/CD Pipeline

- Configure GitHub Actions workflow (`.github/workflows/build.yml`) to compile, test, and generate signed App Bundles (`.aab`) automatically on tag pushes.

---

4. Phase Overview Table

Phase	Core Objective	Key Deliverables	Target Artifacts
Phase 1	Project Setup	Gradle setup, permissions, build config API keys	`build.gradle.kts`, `AndroidManifest.xml`
Phase 2	Camera Viewfinder	Live feed, touch auto-focus, basic reticle view	`CameraPreview.kt`, `ViewfinderOverlay.kt`

Phase	Core Objective	Key Deliverables	Target Artifacts
Phase 3	Cloud MVP	Gemini 1.5 Flash JSON pipeline & detail sheet	`GeminiRepository.kt`, `ComponentDetailSheet.kt`
Phase 4	Edge Inference	TFLite 30 FPS bounding box detection	`ComponentDetector.kt`, `ImageAnalysis` worker
Phase 5	Offline Utilities	Resistor calculator & local Room database cache	`ResistorCalculatorScreen.kt`, Scan Database
Phase 6	Release Hardening	Error boundary, ProGuard, GitHub Actions pipeline	Signed Production `.aab`

---

5. Roo Code / Coding Agent Instructions

When deploying AI coding agents (such as Roo Code) to build or modify the SpecSnap codebase, agents must rigidly adhere to the following workflow principles:

1. **Atomic Step Execution:** Execute exactly one sub-step (e.g., Step 2.2) per prompt/task cycle. Do not attempt to implement complete phases simultaneously.
2. **Git State Validation:** Execute `git status` before starting a task and immediately after completing a task to ensure clean branch tracking.
3. **Build Verification:** Run `./gradlew assembleDebug` after modifying code structures to verify compilation before declaring a task complete.
4. **Architectural Isolation:** Keep UI composables decoupled from inference engines. Pass state via standard state-hoisting paradigms (`StateFlow` / ViewModel state objects).
5. **No Unsanitized Dependencies:** Never introduce third-party libraries not documented in Phase 1 without explicitly notifying project maintainers.